

Protokoll – Tourenplaner Projekt

1. Einleitung

Im Rahmen der Lehrveranstaltung haben wir zu zweit den Tourenplaner umgesetzt. Als Backend haben wir uns für Java/Spring Boot entschieden, als Frontend für Angular mit dem MVVM-Pattern. Die Datenhaltung erfolgt über PostgreSQL mittels Hibernate/JPA als O/R-Mapper. Im Folgenden dokumentieren wir unsere technischen Entscheidungen, aufgetretene Probleme und den aktuellen Stand der Umsetzung.

2. Architektur

Wir haben uns für eine 3-Schichten-Architektur entschieden:

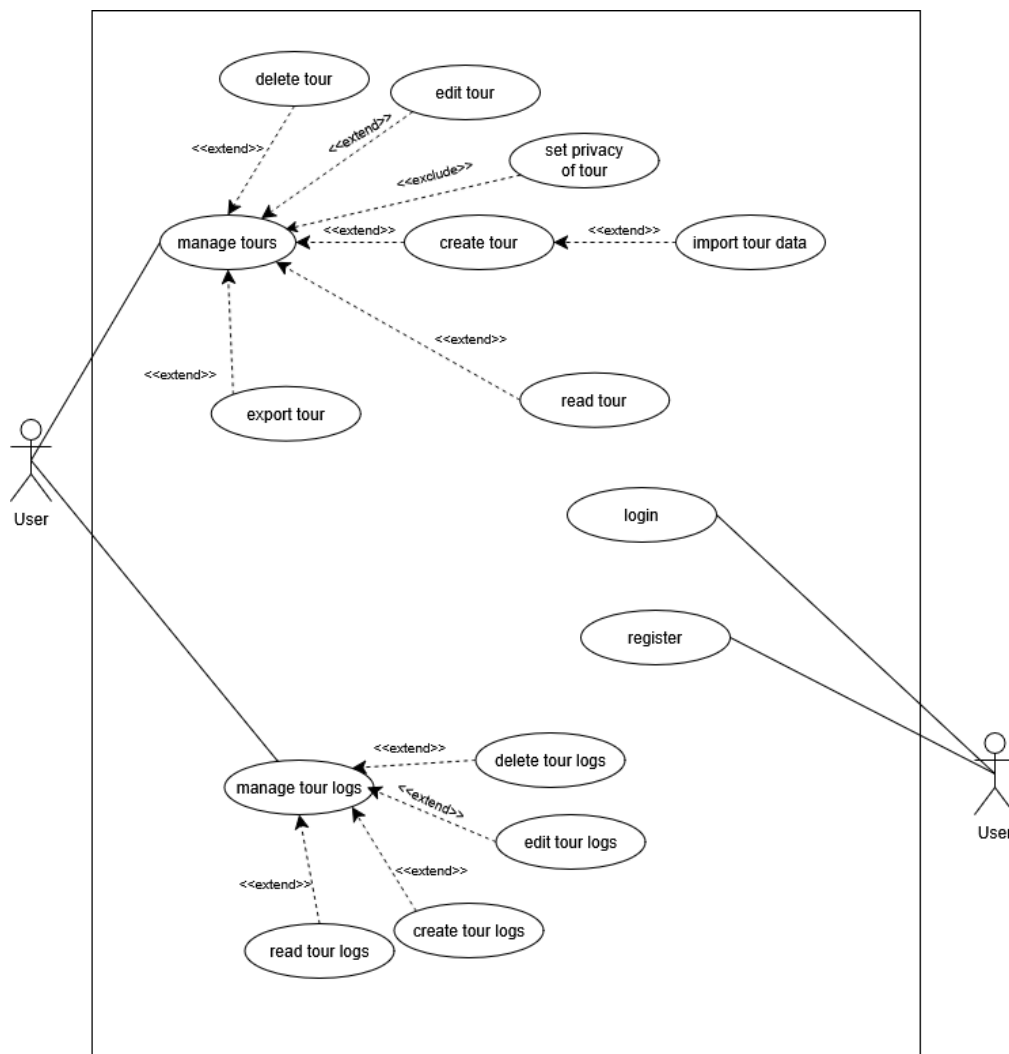
- **Presentation Layer (PL):** Die Controller-Klassen (`TourController`, `TourLogController`, `UserController`, `GlobalTourLogController`) nehmen HTTP-Requests entgegen, validieren über `@Valid` und geben passende HTTP-Statuscodes zurück.
- **Business Layer (BL):** Die Service-Klassen (`TourService`, `TourLogService`, `UserService`, `JwtService`) beinhalten die eigentliche Logik, z.B. Berechtigungsprüfungen (nur der Autor darf seinen TourLog bearbeiten/löschen) oder das Erzeugen von JWT-Tokens.
- **Data Access Layer (DAL):** Die Repositories (`TourRepository`, `TourLogRepository`, `UserRepository`) erben von `JpaRepository` und kapseln den direkten Datenbankzugriff.

Wichtig war uns, dass eine Schicht wirklich nur die direkt darunterliegende Schicht anspricht (Controller ruft nur Services, Services rufen nur Repositories). In den Logeinträgen haben wir das auch mit den Präfixen "PL:" und "BL:" gekennzeichnet, damit man beim Debuggen sofort sieht, aus welcher Schicht ein Log-Eintrag kommt.

Im Frontend haben wir dasselbe Prinzip mit MVVM umgesetzt: Jede Angular-Komponente (View) bekommt ein eigenes injizierbares ViewModel (z.B. [TourLogItemViewModel](#), [TourListViewModel](#)), das den State über Signals hält und die Kommunikation mit den Services übernimmt. Die View selbst enthält dadurch möglichst wenig Logik.

3. Use Cases

In unserem System können User sich registrieren und einloggen, Touren verwalten, Sichtbarkeiten festlegen, Tourdaten import- und exportieren, Touren und Logs erstellen, bearbeiten und löschen.



4. Verwendete Design Patterns

- **Repository Pattern:** Über Spring Data JPA (`JpaRepository`) wird der Datenzugriff abstrahiert, wir müssen kein SQL selbst schreiben, außer bei den Custom-Queries für die Volltextsuche.
- **DTO / Mapper Pattern:** Wir verwenden DTOs (`RequestTourLogDto`, `ResponseTourLogDto`, `AuthDto`) statt die Entities direkt nach außen zu geben, um interne Datenstruktur und API zu entkoppeln. Für TourLogs haben wir dafür MapStruct (`TourLogMapper`) eingesetzt, das uns das Mapping automatisch generiert.
- **Filter Pattern:** `JwtAuthFilter` ist ein klassischer `OncePerRequestFilter`, der vor jeder Anfrage prüft, ob ein gültiges JWT mitgeschickt wurde, und bei Erfolg den `SecurityContext` befüllt.
- **Facade Pattern:** Zb. Für die Leaflet-Integration im Frontend ist als Platzhalter bereits eine `MapFacade`-Klasse angelegt, die die Interna von Leaflet (Karte initialisieren, Marker setzen) kapseln soll, damit die Komponenten selbst nicht direkt mit der Leaflet-API arbeiten müssen.
- **Global Exception Handler (Chain of Responsibility ähnlich):** Über `@RestControllerAdvice` fangen wir alle Exceptions zentral ab und wandeln sie in einheitliche JSON-Fehlerantworten um, ohne dass in jedem Controller Try-Catch-Blöcke nötig sind.

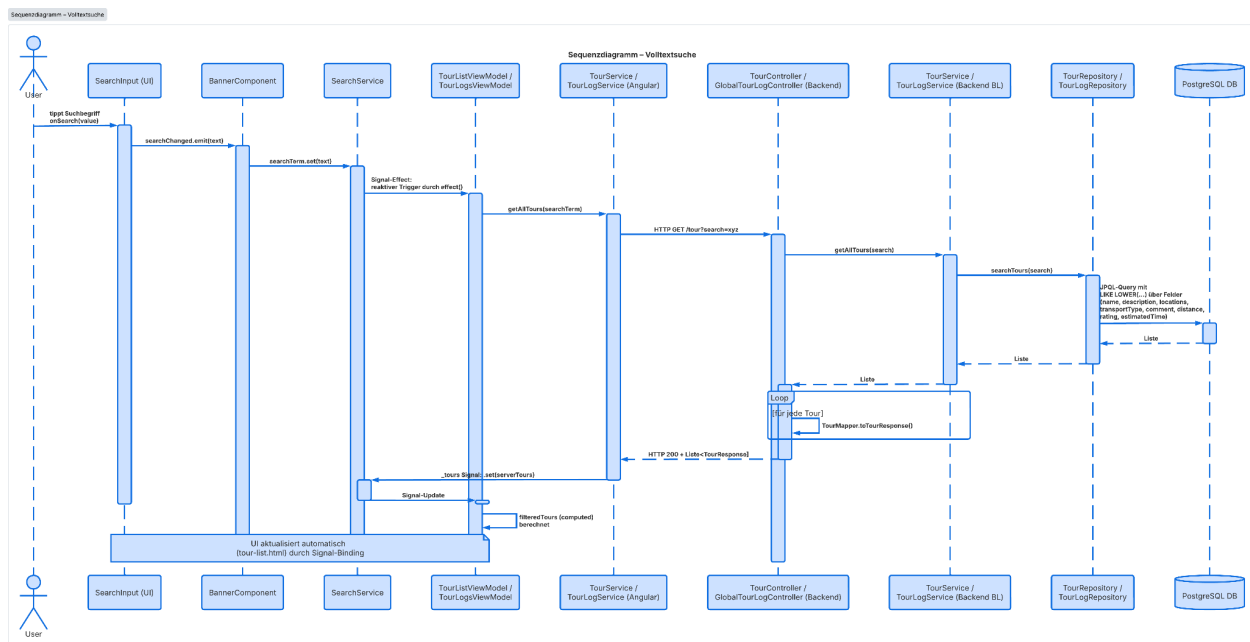
5. Sicherheit / Authentifizierung

Wir verwenden Spring Security mit einem zustandslosen JWT-Ansatz. Nach dem Login (`UserController.login`) wird bei korrektem Passwort (verglichen via `BCryptPasswordEncoder`) ein Token generiert (`JwtService.generateToken`) und im Frontend im `localStorage` abgelegt. Ein Angular-Interceptor (`auth.interceptor.ts`) hängt das Token bei jedem Request automatisch als Authorization-Header an. Wir haben uns bewusst gegen Sessions entschieden, da das Backend so komplett zustandslos bleibt (`SessionCreationPolicy.STATELESS`), was besser zu einer REST-API passt.

Ein Problem, das uns länger beschäftigt hat: Anfangs haben wir versehentlich versucht, den `UserService` direkt im `SecurityConfig` zu injizieren, was zu einem zirkulären Abhängigkeitsproblem geführt hat (`UserService` braucht `PasswordEncoder`, der wiederum in `SecurityConfig` definiert wird). Gelöst haben wir das, indem der `JwtAuthFilter` den `UserService` nur noch `@Lazy` injiziert bekommt.

6. Volltextsuche

Die Volltextsuche haben wir über eigene JPQL-Queries in den Repositories gelöst (`TourRepository.searchTours`, `TourLogRepository.searchByUsernameAndTerm`, `searchByTourIdAndTerm`). Dabei wird über mehrere Felder gleichzeitig gesucht (Name, Beschreibung, Start-/Zielort, Transporttyp, Kommentar der Logs, etc.), case-insensitive über `LOWER()` und `LIKE`. Numerische Felder wie Rating oder Distanz werden über `str()` in Strings umgewandelt, damit man z.B. auch nach "4.5" suchen kann. Auch Attribute wie Child-Friendliness oder Popularity werden miteinberechnet.



7. Reusable UI Component

Im Laufe des Projekts sind mehrere wiederverwendbare Komponenten entstanden, die wir bewusst so gestaltet haben, dass sie über Inputs/Outputs konfigurierbar und

dadurch an mehreren Stellen einsetzbar sind, statt für jede Seite eine eigene Variante zu bauen.

star-rating: Die zentrale Komponente zur Anzeige von Bewertungen. Sie bekommt über ein Input-Signal (**stars**) die Anzahl der Sterne übergeben und rendert entsprechend viele gefüllte bzw. leere Sterne. Verwendet wird sie in der Tourenliste (**tour-card**), auf der Tourdetailseite und bei den TourLogs (**tour-log-item**).

back-button: Eine praktische Komponente, die über den Angular **Location**-Service einfach zur vorherigen Seite zurücknavigiert. Größe (**height**, **width**) und Farbe (**fill**) sind über Inputs konfigurierbar, damit sie sich optisch an den jeweiligen Kontext anpassen lässt. Aktuell wird sie auf der Tourdetailseite verwendet, ließe sich aber problemlos auf weiteren Detailseiten wiederverwenden.

stats-card: Zeigt einen einzelnen Statistik-Wert als Card an, besteht aus einem Bild (**imgPath**), einem Wert (**value**) und einer Beschreibung (**description**). Auf der Stats-Page wird sie mit unterschiedlichen Inputs instanziiert (Anzahl Touren, Gesamtdistanz, Gesamtzeit).

search-input: Kapselt das komplette Such- und Filter-UI (Textsuche plus Dropdown mit Transporttyp, Rating, maximaler Distanz und Dauer) in einer einzigen Komponente. Nach außen kommuniziert sie über die Outputs **searchChanged** und **filterChanged**, sodass die aufrufende Seite selbst nicht wissen muss, wie die Filter-UI intern aufgebaut ist. Eingebunden ist sie in **banner**, wodurch sie auf allen Seiten mit Banner (Tours, TourLogs, MyTours) zur Verfügung steht.

tour-link: Ein wiederverwendbarer "Visit Tour →"-Button. Er bekommt per Input nur die **tourId** und übernimmt intern die Navigation zur Detailseite der Tour. Verwendet wird er z.B. in **tour-log-item**, um von einem Log direkt zur zugehörigen Tour zu springen.

Keine klassische Reusable Component, aber mal erwähnt: **banner** und **sidebar** sind zwar auch eigenständige, gekapselte Komponenten, werden aber jeweils nur einmal pro Layout eingebunden und sind damit eher Layout- als Reusable-Komponenten.

8. OpenRouteService.org Integration

Routenberechnung wird über OpenRouteService.org durchgeführt. Beim Erstellen einer Tour werden Start- und Zielort über die Geocoding-API in Koordinaten umgewandelt. Die Directions-API liefert auch Distanz, Dauer und die vollständige Routengeometrie. Diese Daten werden im Backend verarbeitet, in der Datenbank gespeichert und im Frontend für die Detailansicht und Karte verwendet. So müssen Distanz und geschätzte Dauer nicht manuell eingegeben werden, sondern werden automatisch und konsistent aus echten Routendaten berechnet.

9. Unique Feature: Weather Forecast (Open Meteo)

Unser **unique feature** ist die Wetteranzeige auf der Tourdetailseite. Dafür wird die Open-Meteo-API verwendet, um aktuelle Wetterdaten passend zur Route abzufragen. Die Anwendung bestimmt dafür einen repräsentativen Punkt der Route: Wenn eine Routengeometrie vorhanden ist, wird der mittlere Punkt dieser Geometrie verwendet, ansonsten der geografische Mittelpunkt zwischen Start- und Zielkoordinate. Über den **WeatherService** werden können Daten wie Temperatur, gefühlte Temperatur, Niederschlag, Bewölkung, Windgeschwindigkeit und Wettercode geladen werden. Auf der Tourdetailseite sieht der User dadurch direkt, welche Wetterbedingungen aktuell entlang der Tour zu erwarten sind.

10. Unit Tests

Wir haben uns auf die Bereiche konzentriert, die aus unserer Sicht am kritischsten für die Anwendung sind:

- **TourLogServiceTest:** Hier testen wir die Berechtigungslogik (nur der Autor darf einen Log bearbeiten/löschen) sowie die Zuordnung zwischen Tour und Log. Diese Logik ist besonders wichtig, weil ein Fehler hier direkt dazu führen würde, dass User fremde Daten manipulieren könnten.
- **TourServiceTest:** Grundlegende CRUD-Logik und das korrekte Werfen von **TourNotFoundException**.

- **UserServiceTest:** Testet insbesondere, dass Passwörter vor dem Speichern immer über den **PasswordEncoder** gehasht werden – ein Fehler hier wäre ein sehr großes Sicherheitsproblem.
- **JwtServiceTest:** Token-Erzeugung und -Validierung, mit dem Fall, dass ein manipuliertes oder zu einem anderen User gehörendes Token korrekt abgelehnt wird.
- **GlobalExceptionHandlerTest:** Stellt sicher, dass bei internen Fehlern keine sensiblen Details (z.B. Stacktraces oder interne Exception-Messages) an den Client weitergereicht werden.
- **TourLogMapperTest:** Prüft, dass beim Mapping von Entity zu DTO verschachtelte Objekte (Tour, User) korrekt auf flache Felder abgebildet werden. Würde sonst einfach falsche/leere Daten im Frontend anzeigen.
- **WeatherServiceTests (Angular):** Prüft, ob die Open Meteo API mit den korrekten Koordinaten und Parametern aufgerufen wird und ob die zurückgegebenen Rohdaten korrekt in unser **WeatherInfo**-Modell gemappt werden.

Insgesamt kommen wir aktuell auf über 20 Tests, wobei wir Mockito verwendet haben, um die Schichten isoliert zu testen, ohne eine echte Datenbank zu benötigen.

11. Lessons Learned

- MapStruct hat uns beim TourLog-Mapping viel Schreibarbeit erspart.
- Die Entscheidung, Angular Signals statt klassischem RxJS-State für den globalen Zustand (z.B. **TourService.tours**) zu verwenden, hat den Code lesbarer gemacht, vor allem in Kombination mit **computed()** für abgeleitete Werte wie gefilterte Listen.
- Die zirkuläre Dependency bei JWT/Security hat gezeigt, wie wichtig es ist, sich vorher Überblick von den Abhängigkeiten zwischen Spring Beans zu machen, bevor man einfach drauflos injiziert.
- Commits und Pull Requests sollten klein gehalten werden.

12. Zeitaufwand

Anja Gfrerer

Tätigkeit	Stunden
Setup Backend-Projekt & DB-Anbindung (gemeinsam)	5
Authentifizierung (JWT, Security-Config, Login/Register Backend – inkl. Debugging der zirkulären Dependency)	15
Login- & Registrierungsseite (Frontend)	7
TourLog Backend (Entity, DTOs, Controller, Service, CRUD)	14
TourLog Frontend (TourLogsPage, tour-log-item, tour-popup)	12
Volltextsuche (Backend Queries + Frontend search-input, Filter-Logik)	15
Import/Export von Touren (JSON)	7
Stats-Page (Grundgerüst, stats-card Integration – Backend-Logik noch offen)	5

Unit Tests (TourLogService, JwtService, GlobalExceptionHandler, Mapper)	10
Bugfixing / Debugging (allgemein, u.a. CORS, Interceptor)	9
Protokoll & Dokumentation	7
Summe Anja	106

Eunice Munsi

Tätigkeit	Stunden
Setup Angular-Projekt & Routing-Grundgerüst	5
Tour Backend (Entity, Controller, Service, Repository)	15
Tour Frontend (ToursPage, tour-list, tour-card)	14
MyTours-Seite (Grundgerüst, Import-Button-Anbindung)	7
OpenRouteService-Recherche & erste Integrationsversuche	12
Leaflet-Integration (MapFacade, Recherche)	13
Unique Feature (Umsetzung)	10
Unit Tests (TourService)	5
Styling / Tailwind-Anpassungen allgemein	7
Bugfixing / Debugging (allgemein)	7

Summe Eunice	95
---------------------	-----------

Gesamtaufwand Projekt: ca. 201 Stunden

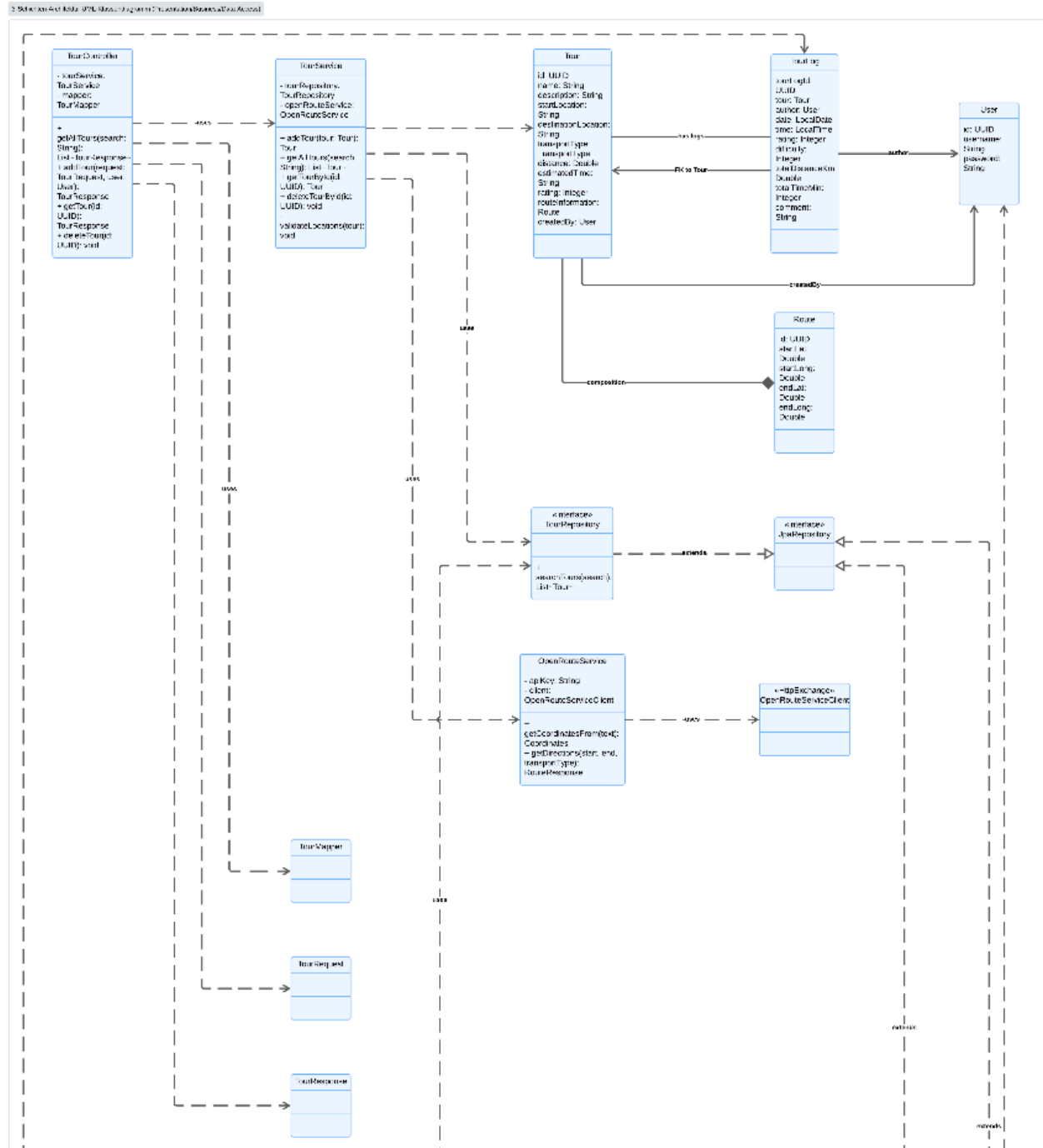
Hinweis: Die Zeiten wurden im Nachhinein grob rekonstruiert, da wir kein durchgehendes Zeit-Tracking-Tool verwendet haben, sondern uns an Git-Commits und eigenen Notizen orientiert haben.

13. Link zum Repository

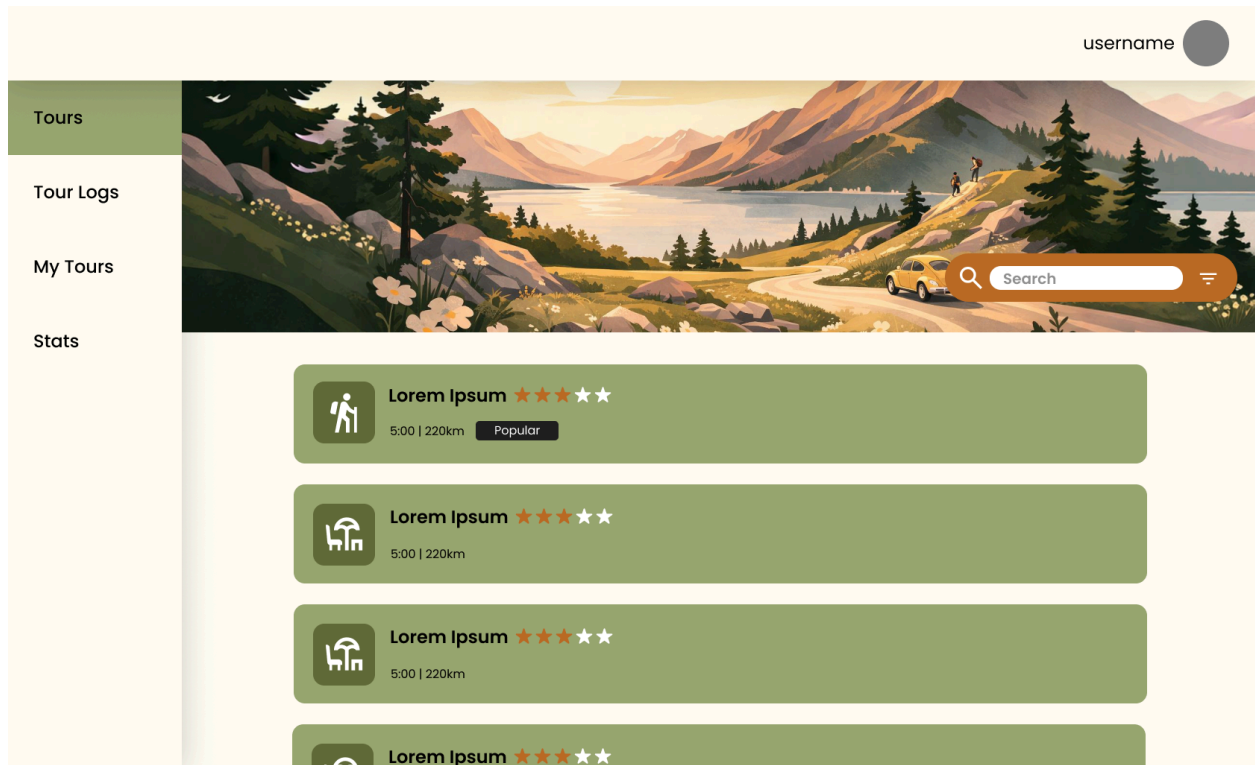
<https://github.com/anjagfrerer/TourPlanner>

13. Sonstige Diagramme

UML – Klassen Diagramm



14. Wireframes / Prototype



Tours

Tour Logs

My Tours

Stats



Lorem Ipsum ★★☆☆☆

5:00 | 220km

Popular



Description:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Tour Logs

Add a Tour Log to this Tour 



Susan Bold
13.01.2026 | 20:25



Difficulty 2/10
Total Distance: 2 km
Total Time: 32 min

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren!



Susan Bold
13.01.2026 | 20:25



Difficulty 2/10
Total Distance: 2 km
Total Time: 32 min

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren!

username

Tours

Tour Logs

My Tours

Stats

My Tour Logs

Susan Bold

13.01.2026 | 20:25

★

★

★

★

★

Difficulty 2/10

Total Distance: 2 km

Total Time: 32 min

Lorem ipsum dolor sit amet, consetetur
sadipscing elitr, sed diam nonumy eirmod tempor
invidunt ut labore et dolore magna aliquyam erat,
sed diam voluptua. At vero eos et accusam et
justo duo dolores et ea rebum. Stet clita kasd
gubergren!

Visit Tour

Susan Bold

13.01.2026 | 20:25

★

★

★

★

★

Difficulty 2/10

Total Distance: 2 km

Total Time: 32 min

Lorem ipsum dolor sit amet, consetetur
sadipscing elitr, sed diam nonumy eirmod tempor
invidunt ut labore et dolore magna aliquyam erat,
sed diam voluptua. At vero eos et accusam et
justo duo dolores et ea rebum. Stet clita kasd
gubergren!

Visit Tour

Susan Bold

13.01.2026 | 20:25

★

★

★

★

★

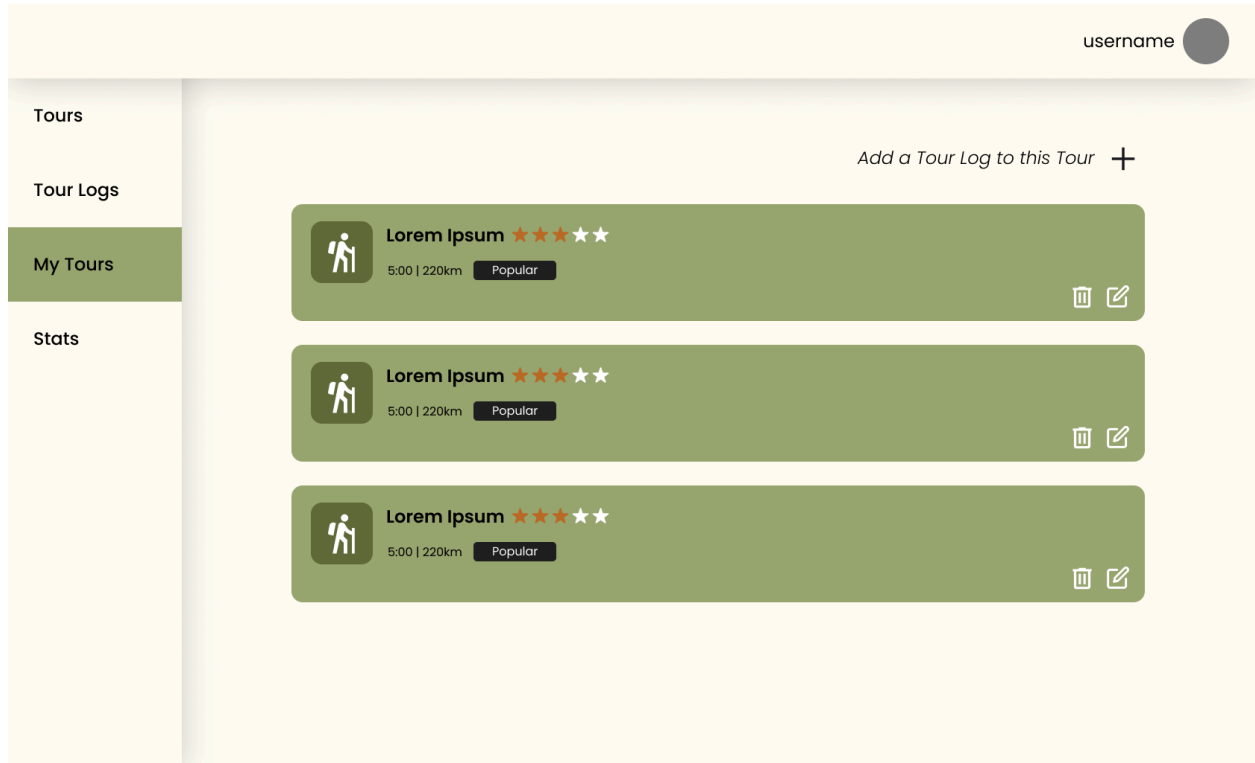
Difficulty 2/10

Total Distance: 2 km

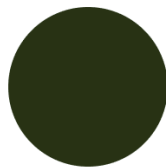
Total Time: 32 min

Lorem ipsum dolor sit amet, consetetur
sadipscing elitr, sed diam nonumy eirmod tempor
invidunt ut labore et dolore magna aliquyam erat,
sed diam voluptua. At vero eos et accusam et
justo duo dolores et ea rebum. Stet clita kasd
gubergren!

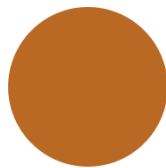
Visit Tour



Primary
#9AA672



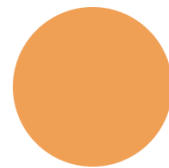
Secondary
#283618



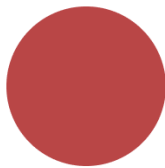
Accent
#bc6c25



Success
#aad576



Warning
#F4A259



Error
#bc4749



Background
#fefae0